# Software Engineering Project Template

## Advanced Academic Style & MDD Showcase

---

**Authors:** SEP4 Group 1

**Date:** April 2026

**Supervisor:** Supervisor Name

**Institution:** VIA University College

# TABLE OF CONTENTS

**1. INTRODUCTION** 1

**2. ANALYSIS ARTIFACTS** 2

2.1 Domain Model 2

2.2 System Sequence Diagram 2

**3. TECHNICAL SPECIFICATION** 3

3.1 Code Implementation 3

3.2 Mathematical Modeling 3

**4. ACADEMIC STANDARDS** 4

**5. REFERENCES** 5

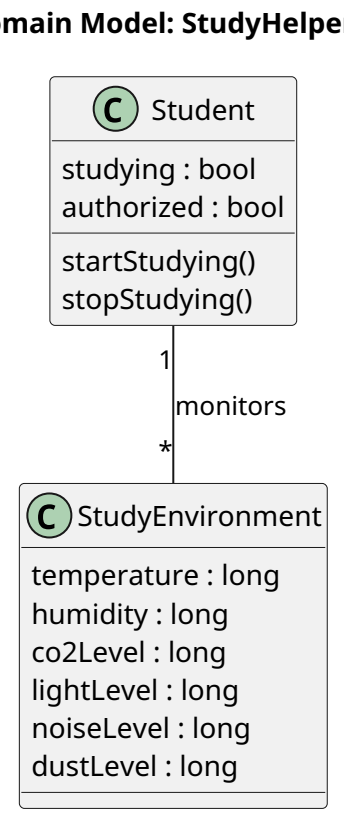
# INTRODUCTION

This document demonstrates the reporting capabilities for the **StudyHelper** project. It integrates Model-Driven Development (MDD) artifacts with advanced academic formatting, including syntax-highlighted code and LaTeX mathematics.

# ANALYSIS ARTIFACTS

## Domain Model

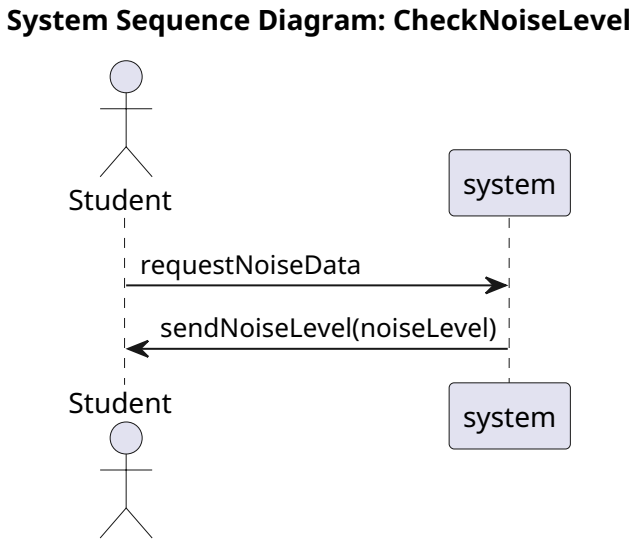
External artifacts generated from `.modt` files are included using standard Markdown syntax. Paths are relative to the document's location in the file system.




StudyHelper Domain Model

# System Sequence Diagram

The following diagram illustrates the interaction for checking environmental noise levels, as defined in the system's analysis models.




Check Noise Level SSD

# TECHNICAL SPECIFICATION

## Code Implementation

The template supports high-contrast syntax highlighting for professional reports. Below is an example of the environmental analysis logic implemented in TypeScript.

```
/**
 * Processes environmental data and checks thresholds.
 */
import { SensorData, Notification } from './types';

export class EnvironmentAnalyzer {
  private readonly CO2_THRESHOLD = 1000;

  public analyze(data: SensorData): Notification | null {
```

```
    console.log(`Analyzing CO2: ${data.co2Level}ppm`);

    if (data.co2Level > this.CO2_THRESHOLD) {
      return {
        type: 'AIR_QUALITY',
        priority: 'HIGH',
        message: 'CO2 level high. Please ventilate the room.'
      };
    }
    return null;
  }
}
```

## Mathematical Modeling

Sensor calibrations and data analysis logic are rendered using professional LaTeX environments.

**CO2 Parts Per Million Calculation:**

\[ \begin{aligned} Ratio &= \frac{V_{out}}{V_{ref}} \\ CO2_{ppm} &= Ratio \times \text{ScaleFactor} + \text{Offset} \end{aligned} \]

# ACADEMIC STANDARDS

## Requirements Mapping

The following table demonstrates the use of semantic status indicators for tracking project progress.

| ID | Requirement | Status | Artifact |
|---|---|---|---|
| R1 | Monitor CO2 Levels | ✔ **Done** | `CheckCO2Level.ssd.svg` |

| ID | Requirement | Status | Artifact |
|---|---|---|---|
| R2 | Notify Student | ⟳ In Progress | `NotificationService.ts` |
| R3 | Data Persistence | ☐ Todo | `PostgreSQL Schema` |

## Citations

- **Parenthetical:** Complex systems require continuous refactoring to maintain quality (Fowler 2018).
- **Narrative:** According to College (2024), software projects must follow a structured lifecycle.

## Task Tracking

- [x] Refactor academic templates for centralized styling.
- [x] Verify LaTeX and Code Highlighting in both HTML and PDF.
- [ ] Implement final sensor drivers.

# REFERENCES


College, VIA University. 2024. "Project Guidelines for Software Engineering." 2024. https://en.via.dk/.

Fowler, Martin. 2018. Refactoring: Improving the Design of Existing Code. 2nd ed. Addison-Wesley Professional.